Program Studi Teknik Informatika
Institut Teknologi Sumatera

IF25-11001 ALGORITMA PEMROGRAMAN

Modul Belajar 14

Rekursi dan Struct

Dari faktorial yang memanggil dirinya sendiri sampai satu wadah untuk NIM, nama, dan IPK

Tim Pengajar

Algoritma Pemrograman. Tahun 2026

Komponen	Keterangan
Pertemuan	14 dari 16
Capaian	Sub-CPMK 14.1 dan 14.2, turunan CPMK 0607 dan 0608
Waktu belajar	sekitar 3 jam belajar mandiri, ditambah 100 menit di kelas
Latihan	`02 Latihan/P14 - Rekursi dan Struct - Latihan/`

Peta belajar

Bagian	Isi
0	Tentang modul ini dan cara memakainya
1	Masalah pembuka
Langkah 1	kasus basis dan langkah rekursif
Langkah 2	trace dengan tumpukan panggilan
Langkah 3	rekursi tanpa basis
Langkah 4	pohon pemanggilan
Langkah 5	menghitung panggilan
Langkah 6	mendefinisikan struct
Langkah 7	menukar elemen struct
Langkah 8	fungsi dengan parameter struct
Langkah 9	kesalahan rekursi dan struct
Penutup	Latihan mandiri, bawa ke tugas, rangkuman, cek pemahaman, bacaan lanjut

Tentang modul ini

Modul ini menemani Pertemuan 14. Isinya sama dengan slide, tetapi ditulis lebih pelan supaya bisa dipelajari sendiri, termasuk oleh yang berhalangan hadir.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Diukur di
14.1	Merancang fungsi rekursif dengan kasus basis dan langkah rekursif yang tepat, serta mendefinisikan struct dan memproses array of struct dengan fungsi	Exit ticket soal 1, Tugas 14, PBL 3, UAS
14.2	Menelusuri pemanggilan rekursif dengan tumpukan panggilan atau pohon pemanggilan, dan menjelaskan peran kasus basis serta akses anggota struct	Exit ticket soal 2, Tugas 14, UAS

Keduanya menurunkan CPMK 0607 dan 0608: menerapkan (0607) dan menjelaskan (0608) konsep dasar algoritma dan sistem komputer untuk menyelesaikan masalah komputasi.

Cara memakai modul ini

Setiap langkah punya kotak merah **Tebak dulu**. Tutup kotak hijau **Jawaban** di bawahnya dengan kertas, tulis dugaan kalian, baru buka. Prosedur penting dijabarkan dalam tiga kolom berdampingan yang dibaca dari kiri ke kanan; sebelum membaca kolom berikutnya, coba tebak dulu langkah apa yang akan datang. Rangkaian setengah jadi lalu mandiri mengurangi bantuan sedikit demi sedikit.

Yang perlu disiapkan

Kertas bergaris, pensil, dan nilai *A* dan *B* dari NIM kalian: *A* digit terakhir (kalau 0, pakai 5) dan *B* digit kedua dari belakang (kalau 0, pakai 3).

Masalah pembuka

PEMBUKA

Folder Tugas berisi berkas dan subfolder, dan setiap subfolder berisi berkas dan subfolder lagi, entah sedalam apa. Menghitung ukuran totalnya dengan perulangan bersarang tidak mungkin, karena banyak lapisnya tidak diketahui. Namun ukuran sebuah folder sama dengan ukuran berkasnya ditambah ukuran setiap subfolder, dan ukuran subfolder dihitung dengan cara yang sama. Itulah rekursi. Masalah kedua datang dari PBL 3: data mahasiswa di tiga array sejajar harus ditukar tiga kali setiap pengurutan. Struct mengumpulkan ketiganya dalam satu wadah.

Langkah 1: kasus basis dan langkah rekursif

REKURSI

Fungsi rekursif adalah fungsi yang memanggil dirinya sendiri dengan masukan yang lebih kecil. Ia selalu punya dua bagian. Kasus basis memberi jawaban langsung tanpa memanggil lagi, misalnya 0! = 1. Langkah rekursif menyatakan jawaban lewat versi masalah yang lebih kecil, misalnya $n! = n \times (n-1)!$, dan masukannya harus bergerak menuju basis.

```
FUNGSI faktorial(n) : integer
  JIKA n = 0 MAKA
    RETURN 1 // basis
  SELAINNYA
    RETURN n * faktorial(n-1)
  AKHIR JIKA
AKHIR FUNGSI
```

Langkah 2: trace dengan tumpukan panggilan

REKURSI

Pemanggilan `faktorial(4)` memanggil `faktorial(3)`, yang memanggil `faktorial(2)`, dan seterusnya sampai `faktorial(0)` mengembalikan 1. Setiap panggilan yang sedang menunggu tersimpan di tumpukan. Lalu nilai kembali naik: 1 × 1 = 1, 2 × 1 = 2, 3 × 2 = 6, 4 × 6 = 24. Ada lima panggilan untuk $n = 4$, secara umum $n + 1$.

Langkah 3: rekursi tanpa basis

REKURSI

Rekursi tanpa kasus basis, atau dengan basis yang tidak pernah tercapai, tidak pernah berhenti dan akhirnya menghabiskan memori tumpukan.

TEBAK DULU

`PROSEDUR hitungMundur(n)` berisi `OUTPUT n` lalu `hitungMundur(n - 1)`. Apa yang terjadi pada `hitungMundur(3)`?

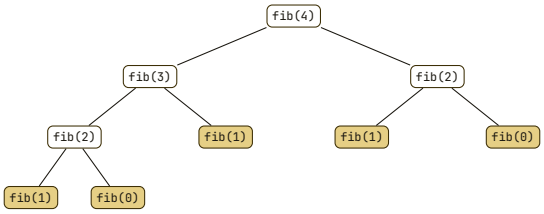
JAWABAN

Tercetak 3, 2, 1, 0, −1, dan seterusnya. Panggilan menumpuk sampai memori tumpukan habis: *stack overflow*. Tambahkan `JIKA n < 0 MAKA RETURN` di awal.

Langkah 4: pohon pemanggilan

REKURSI

Fibonacci memanggil dirinya dua kali, sehingga trace lebih mudah digambar sebagai pohon pemanggilan. Untuk `fib(4)` ada sembilan panggilan, dan `fib(2)` dihitung dua kali. Banyak panggilan tumbuh hampir dua kali lipat setiap n naik satu, sehingga Fibonacci rekursif lambat sekali untuk n besar. Versi iteratif cukup n putaran.



Langkah 5: menghitung panggilan

REKURSI
Banyak panggilan Fibonacci rekursif memenuhi $c(n) = 1 + c(n-1) + c(n-2)$ dengan $c(0) = c(1) = 1$.

TEBAK DULU

`fib(5)` memanggil `fib(4)` dan `fib(3)`. Kalian tahu `fib(4)` butuh 9 panggilan dan `fib(3)` butuh 5. Berapa total panggilan untuk `fib(5)`, dan berapa hasilnya?

JAWABAN

1 + 9 + 5 = 15 panggilan, hasilnya 3 + 2 = 5. Banyak panggilan sendiri mengikuti pola rekursif.

Setengah jadi

Pangkat b^e secara rekursif.

```
FUNGSI pangkat(b, e) : integer
  JIKA e = ____ MAKA
    RETURN ____
  RETURN ____ * pangkat(b, ____)
AKHIR FUNGSI
```

Mandiri

Tulis `FUNGSI jumlahDigit(n) : integer` rekursif, misalnya `jumlahDigit(1234)` = 10. Petunjuk: `n MOD 10` dan `n DIV 10`.

JAWABAN

Setengah jadi: `0`, `1`, `b`, `e - 1`. Mandiri: `JIKA n < 10 MAKA RETURN n`, selainnya `RETURN n MOD 10 + jumlahDigit(n DIV 10)`.

UJI DIRI

Pilih jawaban yang benar. (A) 9, (B) 15, (C) tidak pernah berhenti, (D) 6.

JAWABAN: C

n menjadi 5, 3, 1, −1, −3, dan tidak pernah 0. Basisnya terlewati. A menjumlahkan 5 + 3 + 1 seolah berhenti di 1. B menjumlahkan 1 sampai 5. Untuk $f(4)$ hasilnya memang 6.

Langkah 6: mendefinisikan struct

STRUCT
Struct (record) mengelompokkan beberapa data yang berkaitan menjadi satu tipe baru. Setelah `STRUCT Mahasiswa` didefinisikan, variabel bertipe Mahasiswa memiliki anggota `nim`, `nama`, dan `ipk`, yang diakses dengan titik. Array of struct menyimpan banyak mahasiswa; aksesnya indeks dulu baru titik, misalnya

`mhs[2].nama`.

```
STRUCT Mahasiswa
  nim  : string
  nama : string
  ipk  : real
AKHIR STRUCT
DEKLARASI mhs[40] : Mahasiswa
```

Langkah 7: menukar elemen struct

STRUCT
Variabel struct bisa disalin utuh. Menukar dua elemen array of struct memindahkan semua anggotanya bersamaan.

TEBAK DULU

Saat mengurutkan berdasarkan IPK, dua mahasiswa harus bertukar posisi. Berapa penugasan yang dibutuhkan dengan tiga array sejajar? Berapa dengan array of struct?

JAWABAN

Array sejajar: tiga penukaran, masing-masing tiga penugasan, jadi sembilan. Array of struct: satu penukaran, tiga penugasan, karena `temp` bertipe Mahasiswa dan memindahkan semua anggota sekaligus. Data tidak mungkin tercerai.

Langkah 8: fungsi dengan parameter struct

STRUCT
Array of struct bisa dikirim ke fungsi seperti array biasa. Pencarian berdasarkan NIM sama dengan linear search, hanya pembandingnya yang memakai anggota, `mhs[i].nim`. Pengurutan berdasarkan IPK sama dengan bubble sort, dengan pembanding `mhs[j].ipk` dan satu penukaran elemen utuh.

UJI DIRI

Pilih jawaban yang benar. (A) `mhs[1].ipk ← 3.9`, (B) `mhs.ipk[1] ← 3.9`, (C) `mhs[2].ipk ← 3.9`, (D) `Mahasiswa[1].ipk ← 3.9`.

JAWABAN: A

Indeks dulu (mahasiswa kedua di indeks 1), baru anggota. B memperlakukan ipk sebagai array. C lupa indeks mulai 0. D memakai nama tipe, bukan nama variabel.

UJI DIRI

Pilih jawaban yang benar. (A) 590.000, (B) 90.000, (C) 500.000, (D) 33.000.

JAWABAN: A

90.000 + 250.000 + 250.000 = 590.000, dihitung dari elemen terakhir ke pertama. B hanya elemen terakhir. C lupa elemen terakhir. D menjumlahkan harga saja.

Langkah 9: kesalahan rekursi dan struct

STRUCT
Pada rekursi, periksa bahwa basis tercapai dari setiap masukan, termasuk masukan ganjil dan genap. Pada struct, periksa urutan akses (indeks dulu, baru titik) dan ejaan anggota yang harus sama persis dengan definisi.

Latihan mandiri

Kerjakan lembar latihan di folder `02 Latihan`. Urutannya disusun dari mengingat ke menganalisis.

Tingkat	Waktu	Yang dilatih
1, Dasar	5′	trace pangkat dan definisi struct
2, Menengah	8′	jumlah rekursif dan array of struct
3, Tantangan	7′	memperbaiki tiga fungsi rekursif
Pengayaan	sisa	Menara Hanoi

Kesalahan yang sering muncul

Yang ditulis	Masalahnya	Obatnya
Basis tidak tercapai dari semua masukan	Panggilan tidak berhenti	Uji dengan masukan kecil, ganjil dan genap
Trace hanya sampai panggilan terdalam	Nilai akhir tidak didapat	Lanjutkan naik: kalikan atau jumlahkan saat kembali
`mhs.ipk[i]`	Urutan akses terbalik	`mhs[i].ipk`
`p[N]` pada rekursi array	Di luar batas	`p[N-1]`

Bawa ke tugas

Tugas 14 dikerjakan di kertas: trace `pangkat(3, 4)` lengkap dengan tumpukan panggilan; tulis `jumlahDigit` dan `balikString` secara rekursif beserta basis dan langkahnya; definisikan `STRUCT Karyawan` (nama, jabatan, gaji, lamaKerja) lalu tulis fungsi gaji tertinggi dan rata-rata gaji. Untuk PBL 3, kerjakan Milestone 2.

Rangkuman

Basis yang pasti tercapai, langkah yang memperkecil masalah. Trace dengan tumpukan: turun memanggil, naik kembali. Faktorial $n + 1$ panggilan. Fibonacci rekursif tumbuh hampir dua kali lipat per n; versi iteratif lebih hemat. Satu wadah untuk data yang berkaitan. `mhs[i].ipk`: indeks dulu, baru titik. Menukar satu elemen memindahkan semuanya.

Cek pemahaman

Jawab tanpa membuka modul atau slide. Mengingat kembali tanpa melihat jauh lebih menguatkan ingatan daripada membaca ulang.

No	Soal
1	Apa dua bagian wajib fungsi rekursif?
2	Berapa panggilan untuk faktorial(6)?
3	Mengapa Fibonacci rekursif lambat?
4	Tulis akses nama mahasiswa ketiga di `mhs`.
5	Apa yang terjadi jika basis tidak pernah tercapai?

KUNCI CEK PEMAHAMAN

(1) Kasus basis dan langkah rekursif yang menuju basis. (2) 7. (3) Nilai yang sama dihitung berulang kali; banyak panggilan tumbuh hampir dua kali lipat per n. (4) `mhs[2].nama`. (5) Panggilan menumpuk sampai memori tumpukan habis.

Bacaan lanjut

Gaddis, T. *Starting Out with C++*, Pearson. Deitel, P. dan Deitel, H. *C++ How to Program*, Pearson. Cormen dkk. *Introduction to Algorithms*, MIT Press. Visualisasi interaktif di `visualgo.net`.